

Project 3 — DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely

Type: Multi-agent (3 agents) · **Stack:** Python, LangGraph, RAG (Chroma/FAISS) · **Level:** Intermediate **Tagline:** A 3-agent LangGraph system — Retriever, Reasoner, and Approval — that grounds every answer in your documents and pauses for a human before acting.

The full build spec lives in [10_agent_workflows/README.md](#) (building on [09_langgraph](#)). This file is your workspace and checklist, not a second copy of that spec.

Charter (what this project is)

Project 2's Worker agent, rebuilt as a clear, checkable LangGraph, then grown into three agents working together: a Retriever (searches + judges its own results), a Reasoner (answers only from what was found), and an Approval agent (checks automatically, and only asks a human via `interrupt()` when it really needs to). This is where "agent" becomes "graph," and "one agent" becomes "a small team built right into the graph."

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows an agent whose inner state and decisions are checkable and can be paused/resumed, not a black box — you can point at the exact graph diagram and explain exactly what happens at every step, including pausing for a human before a risky action.
- **Why it matters:** this is the jump from "a script that happens to work" to "a system someone could actually run in production" — clear state, branching, saved state, and human approval are exactly what a hiring manager is checking for when they ask "how would you make this safe to actually use."
- **When you'd build something like this at a real job:** any agent that needs to survive a restart, needs a human check before something important (sending something, spending money, deleting data), or needs to ground its answers in your own documents.
- **How it's built:** a `StateGraph` with typed state, a search step that only runs when needed, saved state, and an `interrupt()` gate before the risky action — a full search → answer → pause → resume → answer cycle you can draw straight from the code.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The graph loops forever, because a branch has no real exit rule	Treat every loop in the graph the same way you'd treat Project 2's step limit — design and test the exit rule on purpose, don't assume it'll "just work"
The state saved at a pause doesn't include what was found, only the original question	Include the found chunks directly in the state shape, so resuming doesn't need to search again or lose its grounding
Search finds nothing relevant, and the graph answers anyway	Route to a clear "can't ground this answer" path when the search results are below a set quality bar, instead of letting it answer without support

Setup (do this once, before Step 1)

```
cd project_3_langgraph_app
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph chromadb python-dotenv py
pip freeze > requirements.txt
```

New here vs. Project 2: `langgraph` (the graph tool), `chromadb` (a local vector store for RAG — swap for `faiss-cpu` if you prefer FAISS). Same `.env` setup as before.

Built During These Documents

[08_rag](#) → [Async Python gate](#) → [09_langgraph](#) → [10_agent_workflows](#)

Plan Before You Code

See [15_five_projects_index](#): write out Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — specifically, sketch the graph's node/edge diagram on paper before writing `graph.py`.

How To Build This — Step by Step

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics

Project: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — Step 1 of 5: A 2-Node Graph With No Branching, Just to Prove the Mechanics

What this step does: proves the basic graph mechanics — state, nodes, one edge — actually work, before any agent behavior or branching gets added. Keeps "do I understand the graph basics" separate from everything after it. **When you'll hit this for real:** the start of every new LangGraph project you'll ever build — always prove the skeleton compiles and runs before adding real logic to it. **Read first:** [09_langgraph Core Concepts](#) — "From a hidden loop to a clear graph," "State," "Nodes and edges."

What to do: 1. Build a minimal 2-node `StateGraph` with a typed state shape and one plain edge — no tools, no branching yet (this is Doc09's own "Basic" exercise). 2. By hand, trace what the state looks like after each node runs. Check that your own understanding matches what the code actually does, before adding anything else.

Your files after Step 1:

```
project_3_langgraph_app/
├─ state.py          → a minimal TypedDict/Pydantic state
├─ nodes.py          → 2 simple node functions
└─ graph.py          → build_graph() with 2 nodes, 1 edge, no branching
```

Step 2 — Project 2's Agent, Rebuilt as a Graph With Saved State

Project: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — Step 2 of 5: Project 2's Agent, Rebuilt as a Graph With Saved State

What this step does: proves your existing agent (from Project 2) can be rebuilt as graph nodes with the exact same behavior, and now gains saved state. **What's new vs. Step 1:** real nodes (Project 2's tool-calling logic) replace the simple placeholder nodes; a conditional edge and a checkpointer get added. **What stays the same:** the tools themselves and the Worker's decisions — you're changing the *container* (loop → graph), not the agent's behavior. **When you'll hit this for real:** any time you outgrow a hand-rolled agent loop and need it to be pausable, resumable, or inspectable — this exact migration, not a rewrite from scratch. **Read first:** [09_langgraph Core Concepts](#) — "Loops," "Checkpointers," "`interrupt()`".

What to do: 1. Rebuild Project 2's Worker agent (tool-calling loop) as graph nodes with a conditional edge — same tools, same behavior, now built as a graph (Doc09's "Real-world" exercise). 2. Connect a checkpointer and prove a pause/resume cycle works on this single-agent graph, before adding any more agents.

Your files after Step 2:

```
project_3_langgraph_app/
├─ state.py
├─ nodes.py          → single-agent tool-calling nodes (from Project 2)
```

— graph.py	→ conditional edge + checkpointer connected
— tools.py	→ reused from project_2_tool_using_agent

Step 3 — A Retriever Agent That Judges Its Own Search Results

Project: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — Step 3 of 5: A Retriever Agent That Judges Its Own Search Results

What this step does: adds the first new specialist — a node that doesn't just fetch chunks, but decides whether what it found is actually good enough to answer from. **What's new vs. Step 2:** a new Retriever node appears, only run when needed; the state grows to carry found chunks, not just the question. **What stays the same:** Step 2's checkpointer and the core graph-building pattern — you're adding a node to an already-proven base, not rebuilding it. **When you'll hit this for real:** any "answer questions about my documents" feature request — this is the exact node that turns a plain agent into a RAG agent. **Read first:** [08_rag Core Concepts](#) — "Vector stores and searching for the top matches."

What to do: 1. Build the **Retriever agent** as its own node: wraps `08_rag`'s `retrieve()`, plus a step that judges relevance (is what came back actually useful? if not, route to a "can't ground this" path instead of forcing an answer downstream). 2. Wire it in as a conditional step in the Step 2 graph — some tasks skip it entirely, same idea as Doc10's routing. 3. Test it: a question the knowledge base can answer, and one it can't — confirm the "can't ground this" path actually runs on the second one.

Your files after Step 3:

project_3_langgraph_app/	
— state.py	→ now includes found chunks, not just the question
— graph.py	→ conditional routing to the Retriever node
— retriever.py	→ reused from 08_rag
— agents/	
— retriever_agent.py	→ search + relevance-judging node
— nodes.py	

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found

Project: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — Step 4 of 5: A Reasoner Agent That Only Answers From What the Retriever Found

What this step does: adds the specialist that actually writes an answer, and only from grounded material — keeping "did we find good context" (Step 3) separate from "did we use it correctly" (this step). **What's new vs. Step 3:** a new Reasoner node uses the Retriever's output; the graph now has search → answer as a real sequence. **What stays the same:** the Retriever from Step 3 is untouched — Step 4 just adds something that uses its output. **When you'll hit this for real:** the

moment a RAG system's answers start drifting from what was actually retrieved — this separation (search quality vs. answer quality) is exactly how you'd diagnose and fix that. **Read first:** [10_agent_workflows Core Concepts](#) — how the pieces fit together.

What to do: 1. Build the **Reasoner agent**: takes the Retriever's output plus the task, and writes a grounded draft answer — clearly told not to answer beyond what was found. 2. Wire search → answer as a real edge sequence. 3. Test it: check that the draft actually reflects the found content, not just believable-sounding text — check this by hand against a couple of test cases, not just "it ran without error."

Your files after Step 4:

```
project_3_langgraph_app/
├─ state.py
├─ graph.py                → search → answer sequence wired up
├─ retriever.py
├─ agents/
│   ├─ retriever_agent.py
│   └─ reasoner_agent.py    → grounded-answer node
└─ nodes.py
```

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents)

Project: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — Step 5 of 5: An Approval Agent That Only Asks a Human When It Really Needs To

What this step does: adds the last specialist — a gate that automatically clears low-risk drafts, and only pauses for a real human when its own checks aren't enough — completing the full 3-agent team. **What's new vs. Step 4:** a new Approval node appears with an `interrupt()` gate; routing now branches based on the approval outcome. **What stays the same:** the Retriever and Reasoner from Steps 3-4 don't change — Approval only watches the Reasoner's draft, it doesn't rewrite it. **When you'll hit this for real:** any agent about to take a consequential, hard-to-undo action — sending something, spending money, deleting data — where full automation is too risky but a human-in-the-loop for every run is too slow. **Read first:** [09_langgraph Core Concepts](#) — "`interrupt()` : pausing for a human", [11_multi_agent_systems Core Concepts](#) — "Sequential" and "Supervisor" patterns.

What to do: 1. Build the **Approval agent**: runs automatic checks on the Reasoner's draft (like — does it actually cite found content, does it avoid a "risky action" without support) — only calls `interrupt()` to ask a real human when its own checks aren't clear, or the action is genuinely important; otherwise it approves on its own and the graph continues. 2. Wire the final routing: task → Retriever (if needed) → Reasoner → Approval → (approved on its own: done) or (interrupt: pause

for a human) → resume → final answer. 3. Test the full cycle, plus the "search found nothing" path (Step 3), plus the "Approval agent approves on its own" path (not every run should need a human).

Your files after Step 5 (final):

```
project_3_langgraph_app/
├─ state.py                → includes found chunks, not just the question
├─ graph.py                → full routing: search? → answer → approval → int
├─ retriever.py            → reused from 08_rag
├─ agents/
│   ├─ retriever_agent.py  → search + relevance-judging node
│   ├─ reasoner_agent.py   → grounded-answer node
│   └─ approval_agent.py   → automatic checks + interrupt() gate
└─ test_project3.py
```

Final Deliverable: DocuMind-RAG-Agent-Ask-Your-Documents-Anything-Safely — a 3-agent LangGraph system. A Retriever judges its own search quality. A Reasoner answers only from grounded material. An Approval agent auto-clears safe drafts and pauses for a human on the rest.

Why this really is multi-agent (said plainly): three separate roles, each a real node (or sub-graph) with its own job and its own pass/fail judgment — not one big node doing everything. There's still no supervisor here (routing is a fixed conditional path, not a dynamic `Command`-based router) — that dynamic routing is Project 4's job.

Checklist Before You Call This Done

- [] `StateGraph` with typed state, at least one real conditional edge
- [] Checkpointer connected — state survives a pause/resume cycle
- [] Retriever agent only runs when needed, not always, and judges its own search results
- [] Reasoner agent produces answers grounded in what the Retriever actually found
- [] Approval agent approves low-risk drafts on its own, and only asks a human via `interrupt()` when needed — both paths tested
- [] Full search → answer → approve/pause → resume → grounded answer cycle works start to finish
- [] You can draw the graph from the code (or the other way around), without running it

Full requirements, test cases, and hints: [10_agent_workflows/README.md](#).

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** through **Hint 4** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**